
EEE3096S Embedded Systems II

Worksheet: Lectures 04 & 05 Benchmarking and Compiler Optimization

Total: 50 marks

Instructions

- This worksheet covers Lecture 04 (Benchmarking) and Lecture 05 (Compiler Optimization).
- Attempt all activities and write your answers in the spaces provided.
- Code answers may be written in C or clear pseudocode. ARM Assembly is not required.
- Marks are shown next to each activity as a guide to depth and pacing.

Question	Available Marks	Grade
1	5	
2	6	
3	5	
4	6	
5	3	
6	7	
7	5	
8	13	
Total	50	

Section A : Benchmarking

Question 1: Golden Measures

[5 marks]

An embedded system is being developed to calculate the distance between two points. Two implementations are available:

- **Implementation A:** a highly optimised C program that executes very quickly.
- **Implementation B:** a slower MATLAB implementation that has been extensively tested and verified to produce numerically correct results.

a) Which implementation would be more appropriate as a Golden Measure? [2]

b) Briefly explain why speed is not the main consideration when selecting a Golden Measure. [3]

Question 2: Benchmarking

[6 marks]

You are comparing two microcontrollers for a battery-powered embedded system. Microcontroller A executes the required program faster than Microcontroller B.

Give **three other factors** that should be considered before deciding that Microcontroller A is the better choice, and briefly explain why each one matters.

1.

2.

3.

Question 3: Benchmarking Pitfalls

[5 marks]

A student benchmarks two implementations by running each program once and recording the wall-clock execution time.

Implementation A: 10 ms Implementation B: 12 ms

The student concludes: *"Implementation A is definitely the better solution because it is faster."*

Do you agree? Explain why or why not. In your answer, identify **at least two issues** with how this benchmark was carried out or interpreted (consider, for example, the effect of running a test only once and whether the system had "warmed up").

Question 4: Standard Benchmarks and Clock Wrapping

[6 marks]

a) The Dhrystone benchmark is a well-known standard benchmark. What type of performance does it test, and state one limitation of relying on it when evaluating a processor for your own application. [2]

b) State briefly what each of the following measures: [2]

MIPS = _____

FLOPS = _____

c) When timing a code section with a high-resolution timer of limited length, the timer value can "wrap" (roll over from a high value back to a low one) during the measurement. Briefly explain how this is handled so the elapsed time is still calculated correctly. [2]

Question 5: Code Review**[3 marks]**

Code reviews are introduced in Lecture 04 as a professional practice for managing code quality.

a) In one sentence, what is the purpose of a code review? **[1]**

b) Name two testing methods mentioned in the lecture that can be used to assess code quality. **[2]**

Section B: Compiler Optimization

Question 6: Compiler Optimization**[7 marks]**

a) What are the three performance optimization invariants that an optimizing compiler should maintain? **[3]**

1.

2.

3.

b) In the expression $\text{execution_time} = \text{NoI} \times \text{CPI}$, what do the following stand for? **[2]**

NoI = _____

CPI = _____

c) Give two reasons why a programmer should still write readable and maintainable code even when using an optimizing compiler. **[2]**

Question 7: GCC Optimization Flags

[5 marks]

The GCC compiler is controlled using command-line optimization flags. Briefly state what each of the following does:

a) -O0 [1]

b) -Os [1]

c) -g [1]

d) In one sentence, what generally changes as you increase the optimization level from -O1 to -O3? [2]

Question 8: Applying Compiler Optimizations

[13 marks]

Part A: Small Function In-lining (SFI)

Consider the following C code:

```
#define OFFSET 5

int addOffset(int z) {
    int m = OFFSET;
    return z + m;
}

int main() {
    int a = readSensor();    // value not known at compile time
    a = addOffset(a);
    return a;
}
```

a) Rewrite main() showing the code after the body of addOffset has been in-lined into main() (bring the function body in; do not simplify yet). [3]

b) Now show the code after any further optimisation that in-lining makes possible (for example, constant folding and removing unnecessary assignments). **[2]**

c) State one benefit and one potential drawback of small-function in-lining. **[2]**

Part B : Loop Unrolling (LUR)

Consider the following C code:

```
for (int i = 0; i < 8; i++) {  
    a[i] = b[i] + 1;  
}
```

- a) Rewrite the loop using loop unrolling by a factor of 2 (two iterations performed per pass).
[3]

- b) State the main benefit of loop unrolling. [1]

-
- c) Why can loop unrolling improve performance? [2]
-